

simple-ring

Mathematical Primitives for Lattice-Based Cryptography

An Educational Implementation for Learning and Experimentation

Table of contents

Introduction	3
Warning	3
What's inside ?	3
How to read it ?	3
ring.rs : The Ring	4
Congruence	4
What is a Ring ?	4
The ring used in cryptography	5
polys.rs : The polynomials in $\mathbb{Z}_q[X]/(X^n + 1)$	6
Polynomial operations	6
Polynomial sum	6
Polynomial subtraction	8
Polynomial multiplication	9
Polynomial scaling	13
Opposite polynomial	13
Polynomial reduction	14
Polynomial division (by a constant)	15
How to use the Polynomial code ?	16
sampling.rs : The coefficient sampling and generation	19
The RNG	19
Implementation for coefficient generation	19
Generation of coefficients with Centered Binomial Distribution (CBD)	20

Generation of small coefficients	22
Uniform generation of coefficients in the Ring	23
Delving into NTT	24
The problem	24
The concept	24
The maths behind NTT	25
What is the root of unity ?	25
What's a primitive root, and why is it important ?	25
The negacyclic	26
The NTT Transform	26
The formal definition of the forward NTT	26
The formal definition of the inverse NTT	27
Proof that $\text{INTT}(\text{NTT}(a)) = a$	28
Proof that $\text{INTT}(\text{NTT}(a) * \text{NTT}(b)) = a * b$	30
The trick to reduce complexity : the butterfly operation	32
<code>modular.rs</code> : modular arithmetic functions and useful ones for searching parameters.	34
Finding the parameters	34
Binary decomposition of the exponent	34
Property of the powers, and computation	34
The modular exponentiation algorithm	35
Finding q	36
The <code>is_prime</code> algorithm	36
The <code>is_q_valid</code> algorithm	37
Finding ω	37
<code>ntt.rs</code> : implementing the NTT, in Rust	39
Twiddle precalculation	39
Forward NTT in single thread	42
Forward NTT in multi-thread	46
Inverse NTT in single-thread	47
Inverse NTT in multi-thread	52
Polynomial multiplication with NTT	54
Errors	55
Glossary	56
References	57

Introduction

This document describes simple-ring, an educational implementation of mathematical primitives for lattice-based cryptography.

Modern post-quantum cryptography (BFV, CKKS, Kyber, Dilithium) relies on the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. Understanding this structure is essential for working with these schemes, but research papers are often complicated, and are often not written for educational purposes, especially in Rust.

Goal: Bridge the gap between abstract mathematics and concrete implementation.

Warning

Before everything, I wanted to warn you about the security of this code. This code **isn't** audited, **isn't** made to be used seriously, **is** vulnerable to timing-based attacks...

So please don't use it in vulnerable environments. But, you can use it to experiment with schemes, you can re-use the NTT, the polynomial code... as you want.

Make everything you imagine with it, but **don't** use it with sensitive data.

What's inside ?

simple-ring provides:

1. Polynomials in $\mathbb{Z}_q$,
2. NTT/INTT for fast multiplication,
3. Modular arithmetic,
4. Sampling methods (CBD, Uniform)

How to read it ?

This documentation revolves around the structure of the crate, and a “progression”. First, we'll introduce mathematical basics, then will explain the Rust code; in details.

ring.rs : The Ring

Congruence

First, let's introduce mathematical basics. The symbol " $\equiv$ " means "is congruent to" ("est congru à" in french), which means :

$$a \equiv b(\text{mod } n) \Leftrightarrow (a \bmod n) = (b \bmod n)$$

For example, if we have $a = 5$, $b = 17$, $q = 12$:

$$a \bmod q = 5 \bmod 12 = 5$$

$$b \bmod q = 17 \bmod 12 = 5$$

So $a \equiv b \bmod q$.

What is a Ring ?

A ring is a mathematical structure where you can :

1. Add elements (integers for example),
2. Subtract elements (addition has inverses),
3. Multiply elements (distributive over additions)

Think of it as a structure where you can define operations and rules.

The formal definition of a commutative ring is a set R equipped with two operations : addition and multiplication.

Property	Addition	Multiplication
Closure	$a + b \in R$	$a * b \in R$
Associativity	$(a + b) + c = a + (b + c)$	$(a * b) * c = (b * c) * a$
Identity	$\exists 0, a + 0 = a$	$\exists 1, a * 1 = a$
Inverses	$\forall a, \exists -a, a + (-a) = 0$	Not always
Commutativity	$a + b = b + a$	$a * b = b * a$
Distributivity		$a * (b + c) = a * c + a * b$

If the multiplication is not commutative, it's a **non-commutative ring**. If there's no multiplicative identity, it's a **pseudo-ring**, and not a ring.

The ring used in cryptography

In lattice-based cryptography, such as **BFV** or **Kyber**, all the computations are done in a commutative ring. It is noted as :

$$\mathbb{Z}_q[X]/(X^n + 1)$$

Where :

$\mathbb{Z}_q$ is a set of integers $\{0, 1, 2, \dots, q - 1\}$ with arithmetic performed modulo q . $\mathbb{Z}_q$ is always a finite commutative ring with q elements. If q is prime, $\mathbb{Z}_q$ is additionally a **field** (every non-zero element has a multiplicative inverse). In the code, all the coefficients are reduced modulo q after each operation.

$[X]$ is the polynomial ring construction : $\mathbb{Z}_q[X]$ denotes the set of all polynomials whose coefficients belong to $\mathbb{Z}_q$. (The form of the polynomial is $a(x) = a_0X^0 + a_1X^1 + \dots + a_kX^k$ where $a_i \in \mathbb{Z}_q$).

Finally, we quotient by the ideal generated by $(X^n + 1)$. In the quotient ring, we have the equality $X^n \equiv -1$. It is useful, because any polynomial of degree $\geq n$ is reduced to a polynomial of degree $< n$ by repeatedly substituting X^n with -1 .

In our Rust code, we implement a structure for the ring :

```
1  //ring.rs 
2  #[derive(Clone)]
3  pub struct RingParams {
4      pub n: usize, //The length of the polynomial, in usize.
5      pub q: u64, //The modulus
6      pub omega: u64, //omega, for the NTT (we'll see it in ntt.rs)
7  }
8  impl RingParams {
9      pub fn new(n: usize, q: u64, omega: u64) -> Self {
10          Self { n, q, omega }
11      }
12  }
```

polys.rs : The polynomials in $\mathbb{Z}_q[X]/(X^n + 1)$

In `simple-ring`, the polynomials “live” in the ring. To implement polynomials in Rust, we simply create a struct `Polynomial` which contains a `Box` of an array of `u64`s :

```
1  pub struct Polynomial {
2      pub coeffs: Box<[u64]>,
3  }
4
5  impl Polynomial {
6      pub fn new(c: Vec<u64>) -> Self {
7          Self { coeffs: c.into_boxed_slice() }
8      }
9
10     pub fn zeros(n: usize) -> Self {
11         Self { coeffs: vec![0u64; n].into_boxed_slice() }
12     }
13 }
```

The `new` function allows you to create a polynomial from a `Vec<u64>`, while the `zeros` function allows you to create an empty polynomial of length `n`, `n` being the value stored in `RingParams`.

Polynomial operations

Now that we have a basic type for the polynomials, we can implement methods for basic polynomial operations, which will be useful when implementing cryptographic schemes.

Polynomial sum

The most simple operation to implement is the polynomial sum in $\mathbb{Z}_q[X]/(X^n + 1)$. The sum is defined by :

$$S_k = A_k + B_k \bmod q$$

Where A and B are two polynomials and S is their sum.
We can easily code it, with this simple function :

```

1  impl Polynomial {
2      #[inline]
3      fn polynomial_sum_single(params: &RingParams, first:
4          &Self, second: &Self) -> Self { //The single-thread sum.
5          let mut coeffs = vec![0u64; params.n];
6          for i in 0..params.n {
7              coeffs[i] = ((first.coeffs[i] as u128 +
8                  second.coeffs[i] as u128) % params.q as u128) as
9                  u64;
10         }
11         Polynomial::new(coeffs)
12     }
13 }

```

In this code, we implement, for the struct *Polynomial*, the function *polynomial_sum_single* which does the sum of two polynomials, and then represent their coefficient modulo q in $\mathbb{Z}_q$. Yes, I could have done something like `coeffs[i].wrapping_add(...)`, or do something like `.iter().zip(...)` but it is less readable than the simple “+” operator, and the goal is here to provide a simple and readable code.

In this crate, I also wanted to implement a parallelized code. So, I used the crate *rayon*, and create a function called *polynomial_sub_multi* which does gives exactly the same result as the *polynomial_sum_single*; but theoretically more faster. In fact, I didn’t notice any difference in term of performances (and sometimes the parallelized version was worst than the single-thread one). Anyway, there is the code for the parallel version :

```

1  impl Polynomial {
2      #[cfg(feature = "parallel")]
3      fn polynomial_sum_multi(params: &RingParams, first:
4          &Self, second: &Self) -> Self {
5          let mut coeffs = vec![0u64; params.n];
6          coeffs
7              .par_iter_mut() //I use the parallelized iter,
8              provided by rayon.
9              .enumerate()
10             .for_each(|(i, coeff)| {
11                 *coeff = ((first.coeffs[i] as u128 +
12                     second.coeffs[i] as u128) % params.q as
13                     u128) as u64;
14             })
15     }
16 }

```

```

10         });
11         Self { coeffs: coeffs.into_boxed_slice() }
12     }
13 }

```

Polynomial subtraction

The polynomial subtraction, in our ring, is noted as :

$$S_k = A_k - B_k \bmod q$$

So it's slightly the same thing as the sum. We write it in Rust :

```

1  #[inline] 🔒 Rust
2  fn polynomial_sub_single(params: &RingParams, first: &Self,
3      second: &Self) -> Self {
4      let mut coeffs = vec![0u64; params.n];
5      for i in 0..params.n {
6          coeffs[i] = ((first.coeffs[i] as i128 -
7              second.coeffs[i] as i128).rem_euclid(params.q as
8              i128)) as u64 ; //We convert into i128 so that we
9              don't have an underflow. The rem_euclid makes the
10              result remain euclidean, so that it is in the ring.
11      }
12      Polynomial::new(coeffs)
13 }

```

Here, the only thing to be aware of is the fact that u64 - u64 can underflow, so that we need to convert it into i128s before computing. Also, because the result of u64 - u64 can be negative, we need to compute .rem_euclid() and not %, so that the result remains euclidean (remains positive). Like for the polynomial sum, I've also implemented a multi-thread function :

```

1  impl Polynomial { 🔒 Rust
2      [cfg(feature = "parallel")]
3      fn polynomial_sub_multi(params: &RingParams, first:
4          &Self, second: &Self) -> Self {
5          let mut coeffs = vec![0u64; params.n];
6          coeffs.par_iter_mut().enumerate().for_each(|(i,
7              coeff)| {

```



```

6         *coeff = (first.coeffs[i] as i128 -
                    second.coeffs[i] as i128).rem_euclid(params.q as
                    i128) as u64;
7     });
8
9     Self { coeffs: coeffs.into_boxed_slice() }
10 }
11 }

```

Polynomial multiplication

Now, let's introduce a more “complex” function : the polynomial multiplication in $\mathbb{Z}_q$. The naive multiplication (not in the ring) is noted as :

$$M_k = \sum_{i+j=k} A_i * B_j$$

Where A, B are two polynomials and M their product. Note that A and B should have exactly the same length.

So, let's implement it !

```

1  impl Polynomial {
2      pub fn polynomial_mul(params: &RingParams, first: &Self,
3                          second: &Self) -> Self {
4          assert_eq!(first.coeffs.len(), params.n); //We
5          ensure the polynomials have the good length, and
6          that they also have the same length.
7          assert_eq!(second.coeffs.len(), params.n);
8
9          let mut b = vec![0u64; 2 * params.n - 2]; //We
10         create a vec, but two times larger (because the
11         product of two polynomials have (2 * n - 2)
12         coefficients.)
13
14         for i in 0..params.n {
15             for j in 0..params.n {
16                 let k = i + j;
17                 b[k] += first.coeffs[i] * second.coeffs[j];
18             }
19         }
20     }
21 }

```

```

14         Self { coeffs: b.into_boxed_slice() }
15     }

```

This “naive” implementation could be good for a simple polynomial multiplication, but doesn’t work for our multiplication in the ring. Also, if n, q are too large, we can have an overflow. So, here are the solutions :

1. To avoid the overflow, we’ll simply use u128 and not u64.
2. To have coefficients in the ring, we just reduce them mod q .
3. But the degree of the result of the multiplication doubles, so we get up to $2n-2$. It means that the result isn’t in $\mathbb{Z}[x]/(X^n + 1)$. For example, let’s have a polynomial $b_{n+k}X^{n+k}$.

This polynomial goes has this form :

$$P(x) = b_0X^0 + b_1X^1 + b_2X^2 + \dots + b_{n+k}X^{n+k}$$

Where k coefficients are “out” of the ring. Here is how to get them back into it :

We use the fact that :

$$X^n + 1 = 0$$

$$X^n = -1$$

So :

$$X^{n+k} = X^n * X^k$$

$$X^{n+k} = (-1) * X^k$$

$$X^{n+k} = -X^k$$

In our case, we have :

$$b_{n+k}X^{n+k} = -b_{n+k}X^k$$

Now, consider the polynomial after multiplication:

$$b(X) = \sum_{i=0}^{2n-2} b_i X^i$$

We split it into two parts : a part that fits in the ring, and a part that is out of it [1]:

$$\sum_{k=0}^{n-1} b_k X^k + \sum_{k=0}^{n-1} b_{k+n} X^{k+n}$$

Using the previous result:

$$b_{k+n} X^{k+n} = -b_{k+n} X^k$$

We have:

$$\sum_{k=0}^{n-1} b_{k+n} X^{k+n} = \sum_{k=0}^{n-1} (-b_{k+n}) X^k$$

We combine both contributions from [1]:

$$b(X) = \sum_{k=0}^{n-1} (b_k - b_{k+n}) X^k$$

$$c_k = b_k - b_{k+n}$$

And in $\mathbb{Z}_q$:

$$c_k = (b_k - b_{k+n}) \bmod q$$

Now, let's implement it in Rust :

```
1  impl Polynomial { Rust
2      #[inline]
3      pub fn mul(&self, params: &RingParams, polynomial:
4          &Polynomial) -> Self {
5          assert_eq!(self.coeffs.len(), params.n);
6          assert_eq!(polynomial.coeffs.len(), params.n);
7
8          let mut b = vec![0u64; 2 * params.n];
9
10         for i in 0..params.n {
11             for j in 0..params.n {
12                 let k = i + j;
13                 b[k] += self.coeffs[i] *
14                     polynomial.coeffs[j] ; //[1]
15             }
16         }
17
18         let mut coeffs = vec![0u64; params.n];
19         for k in 0..params.n { //[2]
20             let low = b[k];
21             let high = if k + params.n < b.len() { b[k +
22                 params.n] } else { 0 };
23
24             let val = ((low as i128) - (high as
25                 i128)).rem_euclid(params.q as i128) as u64;
26             coeffs[k] = val;
27         }
28     }
29     Polynomial::new(coeffs)
```

First, in [1], we do the naive multiplication, and then, in [2], we apply the formula $c_k = (b_k - b_{k+n}) \bmod q$ (low is b_k and high is b_{k+n}). We use `.rem_euclid` so that it remains euclidean (and we cannot have negative

values).

It's done ! The polynomial multiplication is implemented. But we'll see later that this implementation, being too slow (complexity $O(n^2)$), will be replaced¹.

Polynomial scaling

Scaling a polynomial is simply noted as :

$$B_k = (A_k * \lambda) \bmod q$$

Where λ is a constant, q is the modulus, A_k is the polynomial we want to scale and B_k is the result of the scaling. It's easily implementable :

```
1  impl Polynomial { 🔒 Rust
2      #[inline]
3      pub fn scale(&self, params: &RingParams, lambda: u64) ->
        Self {
4          let coeffs: Vec<u64> = self.coeffs
5              .iter() //We iterate for each coefficient
6                  .map(|&coeff| ((coeff as u128 * lambda as u128)
% params.q as u128) as u64) //We pass in u128 so
that it doesn't overflow, then we scale and
reduce by q
7                  .collect::<Vec<u64>>());
8
9          Polynomial::new(coeffs)
10     }
11 }
```

Opposite polynomial

Another “operation” is the one that gives the opposite of a polynomial, as :

$$\hat{A}_k = (-A_k) \bmod q$$

In Rust :

¹We'll see the multiplication in ntt domain, but it is recommended to first see the ntt section before.

```

1  impl Polynomial {
2      #[inline]
3      pub fn opposite(&self, params: &RingParams) -> Self {
4          let q = params.q;
5          let coeffs: Vec<u64> = self.coeffs
6              .iter()
7              .map(|&coeff| ( - (coeff as i128 )).rem_euclid(q
8                  as i128) as u64) //We convert into i128s, so
9                  that it doesn't underflow, then we reduce it.
10             .collect::<Vec<u64>>();
11     }
12 }

```

Polynomial reduction

Another operation is the reduction by a constant. It's noted :

$$\hat{A}_k = A_k \bmod \lambda$$

Where λ is a constant. In Rust, it is written as :

```

1  #[inline]
2  pub fn reduce(&self, constant: u128) -> Polynomial { //
3      Function that reduce the polynomial by a constant (polynomial
4      mod k)
5      let new = self.coeffs
6          .iter()
7          .map(|c| (*c as u128).rem_euclid(constant) as u64) //
8          We consider that the constant is an u128. Currently
9          unused since coefficients are u64, but retained for
10          future extensibility.
11          .collect();
12      Polynomial::new(new)
13 }

```

Polynomial division (by a constant)

The last operation we'll implement in this section is the division by a constant. We write it at :

$$\hat{A}_k = (A_k + \frac{\lambda}{2})/\lambda$$

Why ? Because by default, the compiler does $\lfloor x/y \rfloor$ when dividing non-floating numbers, and not a rounding as $\lfloor x/y \rfloor$. For example :

Operation	In Maths with rounding	In Rust
x/y	$\lfloor x/y \rfloor$	$\lfloor x/y \rfloor$
$7/3$	$\lfloor 2, 33... \rfloor = 2$	$\lfloor 2, 33... \rfloor = 2$
$8/3$	$\lfloor 2, 66... \rfloor = 3$	$\lfloor 2, 66... \rfloor = 2$

When we add $\lambda/2$, we compensate it. For example :

Operation	In Maths with rounding	In Rust
$(x + y)/2)/y$	$\lfloor x/y \rfloor$	$\lfloor x/y \rfloor$
$(7 + 1)/3$	$\lfloor 2, 66... \rfloor = 3$	$\lfloor 2, 66... \rfloor = 2$
$(8 + 1)/3$	$\lfloor 3 \rfloor = 3$	$\lfloor 3 \rfloor = 3$

So, in Rust, we implement :

```
1  #[inline] Rust
2  pub fn divide_by_constant(
3      &self,
4      constant: u128,
5  ) -> Polynomial {
6
7      let new = self.coeffs
8          .iter()
9          .map(|c| {
10              let x = (*c as u128 + constant / 2) / constant;
11              x as u64
12          })
13          .collect();
14      Polynomial::new(new)
15 }
```

How to use the Polynomial code ?

In this section, we'll see how to use the code implemented for polynomials.

Prerequisites :

```
1 cargo add simple-ring
```

 Shell

Then, in your project :

```
1 use simple_ring::{Polynomial, RingParams,
  find_valid_omega, precalculate};
```

 Rust

From this, you can use all the methods defined (code from the lib.rs of simple-ring):

```
1 fn test_polynomials() {
2     let params = RingParams::new(4, 17, find_valid_omega(4,
3     17)); //We define the parameters
4     let ntt_tables = &precalculate(&params);
5     let mut coeffs = vec![0u64; 4]; //We create the
6     coefficients for our first polynomial
7     coeffs[3] = 8;
8     //In all the code, the polynomials should have the same
9     length !
10    let poly1 = Polynomial::new(coeffs.clone()); //We create
11    the first polynomial as P1 = [0, 0, 0, 8]
12    let poly2 = Polynomial::zeros(4); //We create an empty
13    polynomial, which will be the second one.
14    let sum = poly1.sum(&params, &poly2); //We execute the
15    defined methods
16    let mul = poly1.mul(&params, &poly2);
17    let mul_ntt = poly1.mul_ntt(&params, ntt_tables,
18    &poly2);
19    let scaled = poly1.scale(&params, 10);
20    let divided = poly1.divide_by_constant(2);
21    let reduced = poly1.reduce(2);
22    let opposite = poly1.opposite(&params);
23    println!();
24    assert_eq!(poly1.coeffs, coeffs.into_boxed_slice()); //
25    We ensure, with all the assert_eqs, that the result is
26    correct.
27    println!("First polynomial : {:?}", poly1);
```



```

19     println!();
20     println!("Second polynomial : {:?}", poly2);
21     assert_eq!(poly2.coeffs, vec![0u64;
22     4].into_boxed_slice());
22     println!();
23     println!("Sum is : {:?}", sum);
24     assert_eq!(poly1.coeffs, sum.coeffs);
25     println!();
26     println!("Product is : {:?}", mul);
27     assert_eq!(poly2.coeffs, mul.coeffs);
28     println!();
29     println!("Product with NTT is : {:?}", mul_ntt);
30     assert_eq!(poly2.coeffs, mul.coeffs);
31     println!();
32     println!("Scaled first polynomial is : {:?}", scaled);
33     let mut coeffs = vec![0u64; 4];
34     coeffs[3] = (8 * 10) % params.q; //We have to reduce
    because the scale is done modulo q
35     assert_eq!(coeffs.into_boxed_slice(), scaled.coeffs);
36     println!();
37     println!("Divided first polynomial is : {:?}", divided);
38     let mut coeffs = vec![0u64; 4];
39     coeffs[3] = 8 / 2;
40     assert_eq!(coeffs.into_boxed_slice(), divided.coeffs);
41     println!();
42     println!("Reduced first polynomial is : {:?}", reduced);
43     let mut coeffs = vec![0u64; 4];
44     coeffs[3] = 8 % 2;
45     assert_eq!(coeffs.into_boxed_slice(), reduced.coeffs);
46     println!();
47     println!("Opposite poly1 is : {:?}", opposite);
48     let mut coeffs = vec![0u64; 4];
49     coeffs[3] = (-8 as i64).rem_euclid(params.q as i64) as
    u64; //We have to reduce because the opposite is done
    modulo q
50     assert_eq!(coeffs.into_boxed_slice(), opposite.coeffs);
51     println!()
52 }

```

The output is :

```
1  running 1 test
2
3  First polynomial : Polynomial { coeffs: [0, 0, 0, 8] }
4
5  Second polynomial : Polynomial { coeffs: [0, 0, 0, 0] }
6
7  Sum is : Polynomial { coeffs: [0, 0, 0, 8] }
8
9  Product is : Polynomial { coeffs: [0, 0, 0, 0] }
10
11 Product with NTT is : Polynomial { coeffs: [0, 0, 0, 0] }
12
13 Scaled first polynomial is : Polynomial { coeffs: [0, 0, 0,
14  12] }
15
16 Divided first polynomial is : Polynomial { coeffs: [0, 0, 0,
17  4] }
18
19 Reduced first polynomial is : Polynomial { coeffs: [0, 0, 0,
20  0] }
21
22 Opposite poly1 is : Polynomial { coeffs: [0, 0, 0, 9] }
23
24 test test_polynomials ... ok
```

So, the polynomial code is done, we can now approach the sampling !

sampling.rs : The coefficient sampling and generation

In `sampling.rs`, we implement the generation of coefficients for cryptographic implementations.

The RNG

A random number generator (RNG) is essential in cryptography, because if the generated key & noise aren't random (if they are predictable, if they show patterns), the security of LWE is broken. In this project, we've chosen *OsRng*, a random number generator that calls the system generator (`/dev/getrandom` on Linux, for example.). The OS' RNG called is considered as secured because the generated bits depends on physical values like CPU temperature, disk activity, memory used...

So, the first thing to do, in `sampling.rs`, is to define the use of the RNG :

```
1 use crate::{Polynomial, RingParams};
2 use rand::{Rng, RngCore, rngs::OsRng};
```

 Rust

Implementation for coefficient generation

The first thing to do, for implementing the coefficient generation, is to create a struct `Sample`, that will store the coefficients as integers (negative or positive values) :

```
1 #[derive(Clone, Debug)]
2 pub struct Sample(pub Vec<i32>);
3
4 impl Deref for Sample {
5     type Target = Vec<i32>;
6
7     fn deref(&self) -> &Self::Target {
8         &self.0
9     }
10 }
```

 Rust

This struct provides us some modularity : if we want to use values in $\mathbb{Z}$, we can use the struct `Sample`. At the moment we need to get this values

in the ring $\mathbb{Z}_q$, we just reduce each coefficient modulo q .
So, we create a method that implements this transformation :

```
1 impl Sample {
2     pub fn to_poly(self, q:u64) -> Polynomial {
3         let coeffs = self.iter().map(|v| (*v).rem_euclid(q as
4             i32) as u64).collect();
5         Polynomial::new(coeffs)
6     }
```

Generation of coefficients with Centered Binomial Distribution (CBD)

In theory, we should use a gaussian distribution, but it has 2 major problems :

1. The sampling is slow and expensive
2. Its implementation is complicated and source of many errors

So, we want to use something simpler, but with similar properties. Here comes the CBD. We can say that it is a discrete and simple approximation of the gaussian. We define a CBD variable as :

$$X = \sum_{i=1}^{\eta} b_i - b_i'$$

Where $b_i, b_i' \sim \{0, 1\}$

Explanation :

We do the sum of randomly choosen bits, centered on 0, with a base borned in $[-\eta, \eta]$.

Example :

Let's say we want to generate a coefficient, called c . With centered binomial distribution :

1. We choose the value of η . Here, for the example, $\eta = 4$.
2. We generate randomly η bits for b and b' . Here, let's say we have $b = [0, 1, 1, 1]$ and $b' = [0, 1, 0, 0]$.

3. We do the sum of this bits, as $b = 0 + 1 + 1 + 1 = 3$ and $b' = 0 + 1 + 0 + 0 = 1$.
4. We compute the difference between b and b' : $b - b' = 3 - 1 = 2$
5. We have generated our coefficient : $c = 2$.

This method has many good points.

First, it's really fast : we don't call the generator as much as if we had to implement a gaussian distribution, and the operations are really simple (just a sum and a subtraction, trivial for a CPU).

Also, as η grows, the distribution approaches a Gaussian² and, finally, the CBD is subgaussian. It's the perfect candidate for LWE cryptography, such as CRYSTAL-Kyber³ or BFV⁴.

Now, let's implement it :

```
1  #[inline] 🔒 Rust
2  pub fn generate_cbd_sample(n: usize, eta: usize) -> Sample {
3
4      let mut coeffs = vec![0i32; n]; // So first we create an
      empty vector which will contain the coefficients.
5
6      let bits_per_coeff = eta; //Then we choose the number of
      bits for each part (a and b) -> if bit_number == 4, a
      and b will be the sum of 4 bits, like in the example
7
8      let total_bits = 2 * coeffs.len() * bits_per_coeff;
9
10     let total_bytes = (total_bits + 7) / 8;
11
12     let mut rng_buf = vec![0u8; total_bytes];
13
14     OsRng.fill_bytes(&mut rng_buf); //We generate a single
      vector of u8s of which we will choose the bits.
15     let mut bit_index = 0;
16     for coeff in coeffs.iter_mut() {
17         let mut a = 0;
18         let mut b = 0;
```

²https://en.wikipedia.org/wiki/De_Moivre%E2%80%93Laplace_theorem

³<https://pq-crystals.org/kyber/>

⁴please see /schemes/

```

19
    for _ in 0..bits_per_coeff { //Then we choose the
    bits in the sequence of bytes generated before. (for
20    example : generated = [00001111; 10101010; ...],
    then we pick the bits one by one as first bit = 0,
    second one = 0...)
21        let byte = rng_buf[bit_index / 8];
22        let bit = (byte >> (bit_index % 8)) & 1;
23        a += bit as u64;
24        bit_index += 1;
25    }
26
27    for _ in 0..bits_per_coeff { //We continue picking
28        let byte = rng_buf[bit_index / 8];
29        let bit = (byte >> (bit_index % 8)) & 1;
30        b += bit as u64;
31        bit_index += 1;
32    }
33
34    *coeff = a as i32 - b as i32; //And we finally do a
    - b
35    }
36
37    Sample(coeffs) //We return the generated sample
38 }

```

Generation of small coefficients

We will also need to generate small coefficients for our schemes. The generated coefficients will be in the set $\{-1, 0, 1\}$.

We simply implement it :

```

1  #[inline] Ⓜ Rust
2  pub fn generate_small_sample(params: &RingParams) -> Sample
   {
3      let mut rng = OsRng;
4      let mut coeffs: Vec<i32> = Vec::with_capacity(params.n);
5

```

```

6     for _ in 0..params.n {
7         let small: i32 = rng.gen_range(-1..=1); //We
            generate coeffs in the set A = {-1, 0, 1}
8         coeffs.push(small);
9     }
10
11     Sample(coeffs) //We return the generated sample
12 }

```

Uniform generation of coefficients in the Ring

Finally, we need to have a function for generating coefficients uniformly, in the entire ring, so in the set $\{0, 1, \dots, q - 1\}$.

```

1  #[inline] Rust
2  pub fn generate_uniform_polynomial(params: &RingParams) ->
    Polynomial {
3      let mut rng = OsRng; //We choose the rng
4      let mut coeffs = Vec::with_capacity(params.n);
5
6      for _ in 0..params.n {
7          let uniform: u64 = rng.gen_range(0..=params.q -
            1); // We generate values that are in the ring
8          coeffs.push(uniform);
9      }
10     Polynomial { coeffs: coeffs.into_boxed_slice() } //This
        time, we return directly a Polynomial, because we are in
        the ring.
11 }

```

We have now finished with the sampling. Now, we will approach the most difficult concept of this implementation : the NTT.

Delving into NTT

The Number Theoretic Transform is the equivalent of the FFT, but in modular arithmetic : while the FFT is computed on complex numbers and roots of unity as $\omega^n = 1$, the NTT works on a finite field (the ring), with integers modulo q and a primitive root modulo q .

The problem

Let $A(X)$ and $B(X)$ be two polynomials :

$$A(X) = a_0 + a_1 \cdot X^1 + a_2 \cdot X^2$$

$$B(X) = b_0 + b_1 \cdot X^1 + b_2 \cdot X^2$$

Multiplying two polynomials has a complexity of $O(n^2)$, which is enormous for large polynomials. So, we are searching for a representation which reduces the complexity of the operation.

The concept

The NTT says that we can't only represent a polynomial by their coefficients :

$$(a_0, a_1, a_2)$$

$$(b_0, b_1, b_2)$$

But also by the values of the polynomial in some points :

$$(A(\omega^0), A(\omega^1), A(\omega^2))$$

$$(B(\omega^0), B(\omega^1), B(\omega^2))$$

Where ω is a root of unity.

In this representation, we just have to multiply coefficient by coefficient to get the result of the polynomial multiplication.

The maths behind NTT

What is the root of unity ?

In standart arithmetic, a root of unity is a number ω as, for the unity n :

$$\omega^n = 1$$

For example, for $n = 4$, $\omega = i$ because $i^4 = 1$.

But, in the modular arithmetic (in $\mathbb{Z}_q$), the root is a number ω as, for the unit n :

$$\omega^n \equiv 1 \bmod q$$

For example, for $q = 17$ and $n = 4$, $w = 4$ because $4^4 \bmod 17 = 256 \bmod 17 = 1$.

What's a primitive root, and why is it important ?

A primitive root is a root that “generate” all the other ones. A number ω is a primitive root if :

$$\omega^n \equiv 1 \bmod q$$

$$\omega^k \not\equiv 1 \bmod q$$

Where $k < n$.

For example, for $q = 17$ and $n = 4$, $w = 4$ is a primitive root because :

$$4^1 \bmod 17 = 4 \bmod 17 \equiv 4 \not\equiv 1$$

$$4^2 \bmod 17 = 16 \bmod 17 \equiv 16 \not\equiv 1$$

$$4^3 \bmod 17 = 64 \bmod 17 \equiv 13 \not\equiv 1$$

$$4^4 \bmod 17 = 256 \bmod 17 \equiv 1$$

Why do we need a primitive root ?

Because we need $\omega^0 \neq \omega^1 \neq \omega^2 \neq \dots \neq \omega^{n-1}$.

Or, if ω isn't a primitive root, some of the values can repeat, and so we loose information.

The negacyclic

In BFV, we work in the ring $\mathbb{Z}_q[X]/(X^n + 1)$, which means that $X^n \equiv -1 \pmod{(X^n + 1)}$. So, we need $\omega^n \equiv -1 \pmod{q}$.

The solution is to take a $2n$ th root of unity, as :

$$\omega^{2n} \equiv 1 \pmod{q} \Rightarrow \omega^n \equiv -1 \pmod{q}$$

Because $\omega^{2n} = (\omega^n)^2 \equiv 1$, so ω^n is a square root of 1, and, in the ring, the square roots of 1 are $\{-1, 1\}$. If ω is a primitive $2n$ th root, $\omega^n \neq 1$, so $\omega^n = -1$.

Now, we introduce the fact that $q \equiv 1 \pmod{2n}$. Why ? Because a primitive root $2n$ th of unity exists in $\mathbb{Z}_q$ if and **only** if $2n$ divides $(q - 1)$.

For example, if we have $n = 4096$ and $q = 557057$, it works, because :

$$q - 1 \pmod{2n} = 557056 \pmod{8192} = 0$$

In that case, a valid ω exists.

The NTT Transform

The formal definition of the forward NTT

This is the formal definition of the NTT :

Let $a(x)$ be a polynomial of coefficients $[a_0, a_1, \dots, a_{n-1}]$.

The NTT computes :

$$\hat{a}_i = \sum_{j=0}^{n-1} a_j \cdot \omega^{i \cdot j} \pmod{q}$$

And then returns $[\hat{a}_0, \hat{a}_1, \dots, \hat{a}_{n-1}]$.

For example, with $n = 4, q = 17, \omega = 4$, and $a = [1, 2, 3, 0]$, we have :

$$\hat{a}_0 = (a_0 \cdot \omega^0 + a_1 \cdot \omega^0 + a_2 \cdot \omega^0 + a_3 \cdot \omega^0) \pmod{17}$$

$$\hat{a}_0 = (1 \cdot 1 + 2 \cdot 1 + 3 \cdot 1 + 0 \cdot 1) \pmod{17}$$

$$\hat{a}_0 = 6$$

$$\begin{aligned}\hat{a}_1 &= (a_0 \cdot \omega^0 + a_1 \cdot \omega^1 + a_2 \cdot \omega^2 + a_3 \cdot \omega^3) \bmod 17 \\ \hat{a}_1 &= 57 \bmod 17 = 6\end{aligned}$$

$$\begin{aligned}\hat{a}_2 &= (a_0 \cdot \omega^0 + a_1 \cdot \omega^2 + a_2 \cdot \omega^4 + a_3 \cdot \omega^6) \bmod 17 \\ \hat{a}_2 &= 36 \bmod 17 = 2\end{aligned}$$

$$\begin{aligned}\hat{a}_3 &= (a_0 \cdot \omega^0 + a_1 \cdot \omega^3 + a_2 \cdot \omega^6 + a_3 \cdot \omega^9) \bmod 17 \\ \hat{a}_3 &= 75 \bmod 17 = 7\end{aligned}$$

The result is here $\hat{a} = [6, 6, 2, 7]$, we've transform our polynomial into the NTT domain !

But, there is a problem : the computation cost. As you've seen, we have done, for every a , 4 sums and 4 products. So, the complexity is $O(2n^2)$.

The formal definition of the inverse NTT

We've seen the formal definition of the forward NTT. But, to get back into the “normal” coefficients domain, we need an operation called the inverse NTT.

It's formal definition is :

$$a_j = n^{-1} \cdot \sum_{i=0}^{n-1} \hat{a}_i \cdot \omega^{-i \cdot j} \bmod q$$

So, basically, the difference is in the $-i$ (we compute the inverse of $\omega^{i \cdot j}$) and the factor n^{-1} .

For example, with the computed coefficients $\hat{a} = [6, 6, 2, 7]$, we have :

$$\begin{aligned}a_0 &= n^{-1} \cdot (\hat{a}_0 \cdot \omega^0 + \hat{a}_1 \cdot \omega^0 + \hat{a}_2 \cdot \omega^0 + \hat{a}_3 \cdot \omega^0) \bmod 17 \\ a_0 &= n^{-1} \cdot (6 \cdot 1 + 6 \cdot 1 + 2 \cdot 1 + 7 \cdot 1) \bmod 17 \\ a_0 &= \frac{1}{4} \cdot 21 \bmod 17\end{aligned}$$

$$a_0 = 1$$

$$a_1 = n^{-1} \cdot (\hat{a}_0 \cdot \omega^0 + \hat{a}_1 \cdot \omega^{-1} + \hat{a}_2 \cdot \omega^{-2} + \hat{a}_3 \cdot \omega^{-3}) \bmod 17$$

$$a_1 = 2$$

$$a_2 = n^{-1} \cdot (\hat{a}_0 \cdot \omega^0 + \hat{a}_1 \cdot \omega^{-2} + \hat{a}_2 \cdot \omega^{-4} + \hat{a}_3 \cdot \omega^{-6}) \bmod 17$$

$$a_2 = 3$$

$$a_3 = n^{-1} \cdot (\hat{a}_0 \cdot \omega^0 + \hat{a}_1 \cdot \omega^{-3} + \hat{a}_2 \cdot \omega^{-6} + \hat{a}_3 \cdot \omega^{-9}) \bmod 17$$

$$a_3 = 0$$

Done ! We have our original coefficients : $a = [1, 2, 3, 0]$.

Proof that $\text{INTT}(\text{NTT}(\mathbf{a})) = \mathbf{a}$

Now, we want to prove that $\text{INTT}(\text{NTT}(\mathbf{a})) = \mathbf{a}$ in the negacyclic case.
We work with $\omega \in \mathbb{Z}_q$, a primitive **2n-th** root of unity (as seen before):

$$\omega^{2n} \equiv 1 \pmod{q} \quad \text{and} \quad \omega^n \equiv -1 \pmod{q}$$

Step 1: Definitions

NTT:

$$\hat{a}_k = \sum_{j=0}^{n-1} a_j \cdot \omega^{k \cdot j} \bmod q$$

INTT :

$$r_i = n^{-1} \cdot \sum_{k=0}^{n-1} \hat{a}_k \cdot \omega^{-i \cdot k} \bmod q$$

Step 2: Substitution

Substitute $\hat{a}_k$ into the INTT formula:

$$r_i = n^{-1} \cdot \sum_{k=0}^{n-1} \left(\sum_{j=0}^{n-1} a_j \cdot \omega^{k \cdot j} \right) \cdot \omega^{-i \cdot k} \bmod q$$

$$r_i = n^{-1} \cdot \sum_{k=0}^{n-1} \cdot \sum_{j=0}^{n-1} a_j \cdot \omega^{k \cdot j} \cdot \omega^{-i \cdot k} \bmod q$$

$$r_i = n^{-1} \cdot \sum_{k=0}^{n-1} \cdot \sum_{j=0}^{n-1} a_j \cdot \omega^{k \cdot (j-i)} \bmod q$$

Step 3: Orthogonality Lemma

Let $m = j - i$. We analyze the inner sum:

$$S(m) = \sum_{k=0}^{n-1} \omega^{k \cdot m}$$

Case	Value of S(m)
$j = i$ (so $m = 0$)	$\omega^0 = 1$, so $S(0) = \sum_{k=0}^{n-1} 1 = n$
$j \neq i$ (so $m \neq 0$)	Geometric serie, see below

If $j \neq i$, we have a geometric serie :

$$S(m) = 1 + \omega^m + \omega^{2m} + \dots + \omega^{(n-1)m}$$

$$S(m) = \frac{1 - \omega^{n \cdot m}}{1 - \omega^m}$$

Now, since $\omega^{2n} \equiv 1 \pmod{q}$:

$$\omega^{n \cdot m} = (\omega^n)^m \equiv (-1)^m \pmod{q}$$

For m even: $\omega^{nm} \equiv 1$, so numerator = $1 - 1 = 0$.

For m odd: $\omega^{nm} \equiv -1$, but the sum still equals 0 (alternating series $1 - 1 + 1 - 1 + \dots = 0$ for even n). In both cases: $S(m) = 0$ when $j \neq i$.

Step 4: Apply Orthogonality

Only the term where $j = i$ contributes:

$$r_i = n^{-1} \cdot (a_0 \cdot 0 + a_1 \cdot 0 + \dots + a_i \cdot n + \dots + a_{n-1} \cdot 0) \bmod q$$

$$r_i = n^{-1} \cdot a_i \cdot n \bmod q$$

$$r_i = a_i \bmod q$$

Conclusion: We have recovered our coefficient !

Proof that $\text{INTT}(\text{NTT}(\mathbf{a}) * \text{NTT}(\mathbf{b})) = \mathbf{a} * \mathbf{b}$

What we want is to multiply the polynomials in the NTT domain, coefficient by coefficient.

We've said before that the formula of the polynomial multiplication was :

$$c_k = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} a_i b_j \text{ where } i + j \equiv k \pmod{2n}$$

With the constraint $X^n = -1$:

$$r_k = c_k - c_{k+n} \bmod q$$

Where r is the final result of the multiplication.

For explanations and more details, please see the polynomial mul section⁵.

We've also seen the formula of the forward NTT and the inverse NTT. So, let the proof begin :

Let :

- $\hat{a} = \text{NTT}(a)$, $\hat{b} = \text{NTT}(b)$
- $\hat{c} = \hat{a} \odot \hat{b}$

We want to prove that $\text{INTT}(\hat{c}) = a \cdot b$

Let $r = \text{INTT}(\hat{c})$. We have :

⁵poly mul section

$$r_m = n^{-1} \cdot \sum_{k=0}^{n-1} \hat{c}_k \cdot \omega^{-m \cdot k}$$

But $\hat{c}_k = \hat{a}_k \odot \hat{b}$, so :

$$r_m = n^{-1} \cdot \sum_{k=0}^{n-1} \left(\hat{a}_k \cdot \hat{b}_k \right) \cdot \omega^{-m \cdot k}$$

We substitute $\hat{a}$ and $\hat{b}$ by their NTT definitions :

$$r_m = n^{-1} \cdot \sum_{k=0}^{n-1} \cdot \left(\sum_{i=0}^{n-1} a_i \cdot \omega^{k \cdot i} \right) \cdot \left(\sum_{j=0}^{n-1} b_j \cdot \omega^{k \cdot j} \right) \cdot \omega^{-m \cdot k}$$

We can now rearrange the sums :

$$r_m = n^{-1} \cdot \sum_{k=0}^{n-1} \cdot \sum_{i=0}^{n-1} \cdot \sum_{j=0}^{n-1} \cdot a_i \cdot b_j \cdot \omega^{k \cdot i} \cdot \omega^{k \cdot j} \cdot \omega^{-m \cdot k}$$

$$r_m = n^{-1} \cdot \sum_{i=0}^{n-1} \cdot \sum_{j=0}^{n-1} \cdot a_i \cdot b_j \cdot \sum_{k=0}^{n-1} \cdot \omega^{k(i+j-m)}$$

We have, as seen before :

Possibility	Value of the sum
$i + j - m \equiv 0(\text{mod } n)$	$S = n$
$i + j - m \not\equiv 0(\text{mod } n)$	$S = 0$

So the only term where $i + j \equiv m(\text{mod } n)$ contributes. In conclusion, we have :

$$r_m = n^{-1} \cdot \sum_{i=0}^{n-1} \cdot \sum_{j=0}^{n-1} \cdot a_i \cdot b_j \times n \times \delta_{i+j \equiv m}$$

$$r_m = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} a_i b_j \text{ where } i + j \equiv m(\text{mod } n)$$

And then because $\omega^n = -1$, all terms of degree $\geq n$ are automatically reduced. So, the computation of the INTT rebuild exactly the coefficients r_k of $a_k * b_k$ in $\mathbb{Z}_q$:

$$\begin{aligned}
r_k &= \sum_{i+j \equiv k \pmod{n}}^{n-1} a_i \cdot b_j \text{ knowing that } \omega^n = -1 \\
&= \sum_{i+j=k}^{n-1} a_i \cdot b_j - \sum_{i+j=k+n} a_i \cdot b_j \text{ mod } q \\
&= c_k
\end{aligned}$$

The trick to reduce complexity : the butterfly operation

As we've seen, the formal definition of the NTT is :

$$\hat{a}_i = \sum_{j=0}^{n-1} a_j \cdot \omega^{i \cdot j} \text{ mod } q$$

The key is to split even and odd indices. For $n = 4$, let's expand the sum and group the terms :

$$\hat{a}_i = a_0 * \omega^0 + a_1 * \omega^i + a_2 * \omega^{2i} + a_3 * \omega^{3i}$$

We group the indices (0, 2) and (1, 3) :

$$\hat{a}_i = (a_0 * \omega^0 + a_2 * \omega^{2i}) + (a_1 * \omega^i + a_3 * \omega^{3i})$$

We factor out ω^i from the odd part (right) :

$$\hat{a}_i = (a_0 * \omega^0 + a_2 * \omega^{2i}) + \omega^i (a_1 * \omega^0 + a_3 * \omega^{2i})$$

We now define :

$$\text{Evenpart}(i) = a_0 * \omega^0 + a_2 * \omega^{2i}$$

$$\text{Oddpart}(i) = a_1 * \omega^0 + a_3 * \omega^{2i}$$

Therefore :

$$\hat{a}_i = \text{Evenpart}(i) + \omega^i * \text{Oddpart}(i)$$

Then, knowing that $\omega^{i+n/2} = -\omega^i$, we have :

$$\hat{a}_i = \text{Evenpart}(i) + \omega^i * \text{Oddpart}(i)$$

$$\hat{a}_{i+n/2} = \text{Evenpart}(i) - \omega^i * \text{Oddpart}(i)$$

And so, for two coefficients, we only have to compute the same Evenpart, the same Oddpart. We reduce the complexity by two. By computing $\omega^i * \text{Oddpart}(i)$ only one time for the two coefficients, we reduce it again, and, by accessing only one index of coefficients (i) for the results of index i and $i + n/2$, we reduce the memory cost !

Why $O(n \log n)$? Because the butterfly is applied $\log_2(n)$ times :

Level	Butterflies	Work per Butterfly	Total Cost
1	$n/2$	$O(1)$	$O(n)$
2	$n/2$	$O(1)$	$O(n)$
...	...	...	...
$\log_2(n)$	$n/2$	$O(1)$	$O(n)$

In total, we have a cost of $\log_2(n) * O(n) = O(n \log n)$. For exemple, with $n = 4096$, naively we do $4096^2 = 16,777,216$ operations, and with the butterflies we do $n/2 * \log_2(n) = 2048 * 12 = 24,586$ operations, so a gain factor of ~ 682 .

Now, before implementing the NTT code, we will have to implement modular arithmetic functions for searching parameters.

modular.rs : modular arithmetic functions and useful ones for searching parameters.

Finding the parameters

The first thing to do, for the NTT, is to choose the parameters, as $q \bmod 2n = 1$ and ω is a primitive $2n$ th root of n . First, let's implement an algorithm for modular exponentiation.

The concept of the modular exponentiation is that the result of $k^n \bmod q$ is in our ring, in $[0; q - 1]$, so that it doesn't overflow. The problem is that if k and n are enormous, k^n would overflow. The trick is here to avoid computing k^n directly, and to compute $k^n \bmod q$ step by step.

Here, instead of doing $k * k * k * \dots * k$ n times, we use the binary decomposition of the exponent n .

Binary decomposition of the exponent

In binary, the exponent n is written as $n = \sum_{j=0}^l b_j \cdot 2^j$

Where $b_j \in \{0, 1\}$.

For example $n = 13$ is written as 1101 in binary because :

$$1 * 2^0 + 0 * 2^1 + 1 * 2^2 + 1 * 2^3 = 1 + 0 + 4 + 8 = 13$$

Why 1101 and not 1011 ? Because in binary we write from the most significant bit to the less significant. Since $1 * 2^0$ is the less significant and $1 * 2^3$ is the most significant, we write it from $1 * 2^3$ to $1 * 2^0$, so 1101.

Property of the powers, and computation

We have this property :

$$k^n = k^{\sum_{j=0}^l b_j \cdot 2^j}$$

$$k^n = \prod_{j=0}^l k^{b_j \cdot 2^j}$$

$$k^n = \prod_{j=0}^l \left(k^{2^j}\right)^{b_j}$$

The fact is that if $b_j = 0$, $\left(k^{2^j}\right)^{b_j} = 1$, so we can ignore this term, and, if $b_j = 1$, we multiply by k^{2^j} .

So, we can calculate k^{2^l} in an iterative way :

$$\begin{aligned}k^{2^0} &= k \\k^{2^1} &= \left(k^{2^0}\right)^2 = k^2 \\k^{2^2} &= \left(k^{2^1}\right)^2 = k^4 \\k^{2^3} &= \left(k^{2^2}\right)^2 = k^8 \\&\dots\end{aligned}$$

Every step, we do : $k = (k * k) \bmod q$.

The modular exponentiation algorithm

Because of that property, we can implement a step by step function for modular exponentiation in Rust :

```
1  pub fn mod_pow(mut base: u128, mut exp: u128, modulus: u128) -> u128 { 
2      let mut result = 1u128;
3      base %= modulus; //First, we reduce the base with the modulo.
4      while exp > 0 { //Then, while the exponent is not equal to 0, we continue the exponentiation
5          if exp & 1 == 1 { //If the less significant bit is 1, we multiply
6              result = (result * base) % modulus;
7          }
8          base = (base * base) % modulus; //We prepare the base for the next iteration
9          exp >>= 1; //We pass to the next bit. (so that the next bit becomes the less significant bit for next step)
10     }
11     result //We return the result of the exponentiation
12 }
```

This method avoids the overflow, and is fast, because we have at most $\log_2(\text{exp})$ iterations. Since exp is at the most of the time an u64 in our code, $\log_2(2^{64}) = 64$ iterations maximum.

Finding q

We'll not implement a function for searching q , because it depends on the scheme we choose. For example, q in BFV is totally different from q in CKKS (in BFV, we'll not implement the RNS⁶, but in CKKS we'll not have the choice).

But, we can already implement an algorithm that returns a boolean, and that check if q is compatible with the NTT or not. First, let's implement an algorithm that tests if a number is prime or not.

The `is_prime` algorithm

```
1  fn is_prime(n: u64) -> bool { 🔒 Rust
2      if n < 2 { return false; }      // 0, 1 aren't primes
3      if n == 2 { return true; }      // 2 is prime, and the
        only even prime
4      if n.is_multiple_of(2) { return false; } // All other
        even numbers aren't prime
5
6      // Test divisors from 3 to  $\sqrt{n}$ 
7      let mut i = 3u64;
8      while i * i <= n {                // Condition :  $i \leq \sqrt{n}$ 
9          if n.is_multiple_of(i) {      // if i divide n
10             return false;             // n isn't a prime
                number
11         }
12         i += 2;                        // Next one
13     }
14     true // If no divisor was found, n is prime
15 }
```

Why do we test to $\sqrt{n}$? Because if n as a divisor $d > \sqrt{n}$, $\frac{n}{d} < \sqrt{n}$ is also a divisor.

Proof :

Let d be a divisor of n as $d > \sqrt{n}$. We have $n = d * k$.

If $k > \sqrt{n}$, $d * k > \sqrt{n} * \sqrt{n} = n$. But it isn't possible, since d is a divisor

⁶https://en.wikipedia.org/wiki/Residue_number_system

of n .

So $k \leq n$, and so we just have to test the divisors from 3 to $\sqrt{n}$.

The `is_q_valid` algorithm

Now, we can implement the algorithm :

```
1  pub fn is_q_valid(n: usize, q: u64) -> bool {  Rust
2      if n == 0 || q < 2 { //If q < 2, or n = 0, then it can
          not be valid
3          return false;
4      }
5
6      if !is_prime(q) { //if it is not prime, it cannot be
          valid
7          return false;
8      }
9
10     let two_n = 2 * n as u64; //If the condition q - 1 mod
        2n == 0 is true, then q is valid
11     (q - 1) % two_n == 0
12 }
```

Finding ω

Now that we have q , `is_prime`, and n , we can find ω as a primitive $2n$ th root of n :

```
1  pub fn find_valid_omega(n: usize, q: u64) -> u64 {  Rust
2      let two_n = 2 * n;
3      let exp = (q - 1) / two_n as u64;
4
5      for candidate in 2..q {
6          let omega = mod_pow(candidate as u128, exp as u128,
              q as u128) as u64;
7
8          let omega_n = mod_pow(omega as u128, n as u128 , q
              as u128);
9          if omega_n == (q as u128 - 1) {
10             return omega;
          }
      }
  }
```

```

11     }
12 }
13 panic!("No valid omega found");
14 }

```

Why does $e = (q - 1)/2n$? :

The function needs to return an integer so that ω is a primitive $2n$ th root of unity in $\mathbb{Z}_q$:

1. $\omega^{2n} \equiv 1(\text{mod } q)$
2. $\omega^n \equiv -1(\text{mod } q)$
3. $\omega^k \not\equiv 1(\text{mod } q)$ for each $k < 2n$

We know that $q \equiv 1(\text{mod } 2n)$, so $e = (q - 1)/(2n)$ is an integer.
For each candidate $c \in [2, q - 1]$:

$$\omega = c^e \text{ mod } q$$

Because of Fermat's little theorem⁷, we have :

$$c^{q-1} \equiv 1(\text{mod } q)$$

So :

$$\omega^{2n} = (c^e)^{2n}$$

$$\omega^{2n} = c^{e*2n}$$

$$\omega^{2n} = c^{q-1}$$

$$\omega^{2n} \equiv 1(\text{mod } q)$$

It works !

⁷https://en.wikipedia.org/wiki/Fermat%27s_little_theorem

ntt.rs : implementing the NTT, in Rust

Now, we can - finally - implement the NTT in Rust. First, we'll detail the *twiddle precalculation*.

Twiddle precalculation

A *twiddle* is a precomputed power of ω , as :

$$t_k = \omega^k \bmod q$$

It's called twiddles because in FFT and NTT, the values are “twiddled” (I think it's the reason, but maybe I'm wrong).

We have :

$$r(x) = \text{bit_reverse}(x)$$

$$t[k] = \omega^{r(k)} \bmod q$$

$$\overline{t}[k] = \omega^{-r(k)} \bmod q$$

$$\psi = \omega$$

$$\overline{\psi} = \omega^{-1} \bmod q$$

Where not overlined means for forward NTT, and overlined means for inverse NTT.

We have a bit-reversed order because the “Merge in place” algorithm access to the twiddles in an order that correspond to bit_reversal of indices (We'll see it later). Instead of calculating bit_reverse(k) every time, we precompute it in bit-reversed order, so that the bit_reverse is done one time, and not $O(n \log(n))$ times.

So, to precompute the twiddles, we first create a structure called :

```

1 use crate::RingParams;
2 use crate::modular::mod_pow;
3 use crate::polys::Polynomial;
4
5 #[derive(Clone)]
6 pub struct NTTprecaculated {
7     twiddles: Box<u128>, //Storage for twiddles
8     twiddles_inv: Box<u128>, //Storage for inv_twiddles
9 }

```

Then, we implement the precalculation of the twiddles :

```

1 pub fn precalculate(params: &RingParams) ->
  NTTprecaculated {
2     let n = params.n;
3     let q = params.q as u128;
4     let psi = params.omega as u128; //Yes, it's just a name
    changing, but it's how it's written in the formula.
5     let log_n = n.trailing_zeros(); //Because n is a power
    of 2, the number of trailing zeros gives us log2(n)
6
7     let mut twiddles = vec![u128; n];
8     for k in 0..n {
9         let br_k = k.reverse_bits() >> (64 - log_n); //
    Reverse bit of k on logn bits
10        let power = br_k as u128;
11        twiddles[k] = mod_pow(psi, power, q); //The twiddle
    is psibit_reversed(k)
12    }
13
14
15    let psi_inv = mod_pow(psi, q - 2, q); //because of
    fermat's little theorem
16    let mut twiddles_inv = vec![u128; n];
17    for k in 0..n {
18        let br_k = k.reverse_bits() >> (64 - log_n);
19        let power = br_k as u128;
20        twiddles_inv[k] = mod_pow(psi_inv, power, q);

```



```

21     }
22
23     NTTprecaculated {
24         twiddles: twiddles.into_boxed_slice(), //We return
           the twiddles
25         twiddles_inv: twiddles_inv.into_boxed_slice(), //And
           the twiddle inv
26     }
27 }

```

What are trailing zeros ? Trailing zeros are non-significants zeros in binary representation. For example, with $n = 4096$, we have :

$$n = 4096$$

$$n = 2^{12}$$

$$n = 1000000000000$$

There are 12 trailing zeros, and $\log_2(4096) = 12$.

Why using $\log_2(n)$? Because we do the bit_reverse on exactly $\log_2(n)$ bits. For example, with $n = 8$:

$$n = 2^3$$

$$\log_2(2^3) = 3$$

$$k = 1 = 001 \Rightarrow r(k) = 100 = 4$$

$$k = 2 = 010 \Rightarrow r(k) = 010 = 2$$

$$k = 3 = 011 \Rightarrow r(k) = 110 = 6$$

Forward NTT in single thread

The NTT used here is a modern one from this paper : Homomorphic encryption on GPU, 2022/1222⁸.

```
1  fn forward_ntt_single( 🔒 Rust
2      params: &RingParams, //We'll use the defined parameters,
3      polynomial: &Polynomial, //The polynomial to convert in
        NTT domain,
4      ntt_tables: &NTTprecalculated //And the precalculated
        twiddles.
5  ) -> Polynomial {
6      let mut coeffs: Vec<u64> = polynomial.coeffs.to_vec();
7
8      let n = params.n;
9      let q = params.q as u128; //Conversion for later
10     let mut t: usize = n;
11     let mut m: usize = 1;
12
13     loop {
14         t /= 2; //Distance between 2 butterflies
15         for i in 0..m { //For every group of butterflies
16             let j1 = 2 * i * t;
17             let j2 = j1 + t - 1;
18
19             for j in j1..=j2 {
20                 let u = coeffs[j] as u128; //The butterfly
                    operation
21                 let w = ntt_tables.twiddles[m + i]; //We use
                    the precomputed twiddles
22                 let v = ((coeffs[j + t]) as u128 * w) %
                    q; //Reduction so that it remains in the
                    ring
23
24                 let mut x = u + v; //High (a_i)
25                 if x >= q { x -= q; } //reduction
```

⁸Özcan, Ali & Ayduman, Can & Türkoğlu, Enes & Savaş, Eray. (2023). Homomorphic Encryption on GPU. IEEE Access. PP. 1-1. 10.1109/ACCESS.2023.3265583.

```

26         coeffs[j] = x as u64; //We insert the
           coefficient
27
28         let y = if u >= v { u - v } else { u + q
           - v }; //Low (a_(i+t)) then reduction
29         coeffs[j + t] = y as u64; //We insert
           the coefficient
30
31     }
32 }
33
34     m *= 2;
35
36     if m >= n { break; } //To stop the loop, when it's
           finished
37 }
38
39 Polynomial {
40     coeffs:
41     coeffs.into_iter().collect::<Vec<_>>().into_boxed_slice()
           We return the coefficients, into a Polynomial.
41 }
42 }

```

Explanation :

The t iteration is done so that we get the coefficients that are in pair :

$t=4$: [0] $\longleftrightarrow$ [4] [1] $\longleftrightarrow$ [5] [2] $\longleftrightarrow$ [6] [3] $\longleftrightarrow$ [7]

$t=2$: [0] $\longleftrightarrow$ [2] [1] $\longleftrightarrow$ [3] [4] $\longleftrightarrow$ [6] [5] $\longleftrightarrow$ [7]

$t=1$: [0] $\longleftrightarrow$ [1] [2] $\longleftrightarrow$ [3] [4] $\longleftrightarrow$ [5] [6] $\longleftrightarrow$ [7]

The m iteration is done so that all butterflies that uses the same twiddle factor uses the twiddle factor of index m . We do $m = m * 2$ because each time we divide the distance between two butterflies (t), we scale by 2 the number of groups.

j_1 and j_2 are the bounds of the group :

$$j_1 = 2 * i * t \rightarrow \text{index of the first bound}$$

$$j_2 = j_1 + t - 1 \rightarrow \text{index of the second bound}$$

And j is the index of u in each butterfly.

For example, with $n = 8$, at the second iteration ($t = 2, m = 2$) :

Group $i = 0$:

$$j_1 = 2 * 0 * 2 = 0$$

$$j_2 = 0 + 2 - 1 = 1$$

Butterflies on $\text{coeffs}[j] \leftrightarrow \text{coeffs}[j + t]$ and $\text{coeffs}[j] \leftrightarrow \text{coeffs}[j + t]$.

Butterflies on $\text{coeffs}[0] \leftrightarrow \text{coeffs}[2]$ and $\text{coeffs}[1] \leftrightarrow \text{coeffs}[3]$.

Group $i = 1$:

$$j_1 = 2 * 1 * 2 = 4$$

$$j_2 = 4 + 2 - 1 = 5$$

Butterflies on $\text{coeffs}[j] \leftrightarrow \text{coeffs}[j + t]$ and $\text{coeffs}[j] \leftrightarrow \text{coeffs}[j + t]$.

Butterflies on $\text{coeffs}[4] \leftrightarrow \text{coeffs}[6]$ and $\text{coeffs}[5] \leftrightarrow \text{coeffs}[7]$.

...

We can visualize like this, for example :

Initial state : $\text{coeffs} = [a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7]$.

Iteration 1 : $t = 4, m = 1$:

Group $i = 0$: $j_1 = 0, j_2 = 3$

$j=0$: $\text{coeffs}[0] \leftrightarrow \text{coeffs}[4]$ (twiddle = twiddles[1])

$j=1$: $\text{coeffs}[1] \leftrightarrow \text{coeffs}[5]$ (twiddle = twiddles[1])

j=2 : coeffs[2] $\leftrightarrow$ coeffs[6] (twiddle = twiddles[1])

j=3 : coeffs[3] $\leftrightarrow$ coeffs[7] (twiddle = twiddles[1])

4 butterflies, same twiddle, distance of 4.

Iteration 2 : $t = 2$, $m = 2$:

Group $i = 0$: $j_1 = 0$, $j_2 = 1$:

j=0 : coeffs[0] $\leftrightarrow$ coeffs[2] (twiddle = twiddles[2])

j=1 : coeffs[1] $\leftrightarrow$ coeffs[3] (twiddle = twiddles[2])

Group $i = 1$: $j_1 = 4$, $j_2 = 5$

j=4 : coeffs[4] $\leftrightarrow$ coeffs[6] (twiddle = twiddles[3])

j=5 : coeffs[5] $\leftrightarrow$ coeffs[7] (twiddle = twiddles[3])

2 groups of 2 butterflies, distance of 2

Iteration 3 : $t = 1$, $m = 4$

Group $i = 0$: $j_1 = 0$, $j_2 = 0 \rightarrow j = 0$: coeffs[0] $\leftrightarrow$ coeffs[1] (twiddles[4])

Group $i = 1$: $j_1 = 2$, $j_2 = 2 \rightarrow j = 2$: coeffs[2] $\leftrightarrow$ coeffs[3] (twiddles[5])

Group $i = 2$: $j_1 = 4$, $j_2 = 4 \rightarrow j = 4$: coeffs[4] $\leftrightarrow$ coeffs[5] (twiddles[6])

Group $i = 3$: $j_1 = 6$, $j_2 = 6 \rightarrow j = 6$: coeffs[6] $\leftrightarrow$ coeffs[7] (twiddles[7])

4 groups of 1 butterfly, distance of 1

Result : coeffs = [$\hat{a}_0$, $\hat{a}_4$, $\hat{a}_2$, $\hat{a}_6$, $\hat{a}_1$, $\hat{a}_5$, $\hat{a}_3$, $\hat{a}_7$] (order bit-reversed)

Forward NTT in multi-thread

This NTT is also based on the research paper, but implements parallelism with rayon.

```
1  #[cfg(feature = "parallel")] Rust
2  fn forward_ntt_multi(
3      params: &RingParams, //Same, the use the parameters, the
        polynomial and the precalculated twiddles.
4      polynomial: &Polynomial,
5      ntt_tables: &NTTprecalculated
6  ) -> Polynomial {
7      let mut coeffs: Vec<u128> = polynomial.coeffs
8          .iter() //We iter, and collect, it's the same thing
        as in the forward code, but written differently and
        passing it into u128s
9          .map(|&c| c as u128)
10         .collect();
11
12     let n = params.n;
13     let q = params.q as u128;
14
15     let mut t: usize = n; //The same
16     let mut m: usize = 1;
17
18     loop {
19         t = t / 2;
20
21         coeffs
22             .par_chunks_mut(2 * t) //We do a par_chunk_mut, which is
        an operation implemented by rayon which allows us to
        "cut" the vector into chunks of a size of 2t
23         .enumerate()
24         .for_each(|(i, chunk)| { //We treat each chunk in
        parallel. It's the only thing that changes
25             for j in 0..t {
26                 let u = chunk[j];
27                 let w = ntt_tables.twiddles[m + i];
28                 let v = ((chunk[j + t]) * w) % params.q as u128;
```

```

29         let sum = u + v;
30
31         chunk[j] = if sum >= q { sum - q } else
            { sum };
32
33         chunk[j + t] = if u >= v { u - v } else { u + q
            - v };
34     }
35 });
36
37
38     m = m * 2;
39
40     if m >= n { break; }
41 }
42
43 Polynomial {
44     coeffs: coeffs.into_iter().map(|c| c as
        u64).collect::<Vec<_>>().into_boxed_slice(), //Same
        thing as in the single-thread code, except the fact
        that we repass it into u64s
45 }
46 }

```

There is no much to say about this function, except the fact that we use a parallelized iteration.

Inverse NTT in single-thread

Now that we have our forward NTT function, we need to introduce the inverse NTT function. It's basically the same, except that we use inverse twiddles ($\bar{t}[k]$) and that we scale by n^{-1} at the end of the function. The INTT used is also from the *Homomorphic Encryption on GPU* paper.

```

1  fn inverse_ntt_single(
2      params: &RingParams, //We use the defined parameters,
3      polynomial: &Polynomial, //The polynomial to get back
        into "normal" domain

```



```

4      ntt_tables: &NTTprecalculated //The precalculated tables
5  ) -> Polynomial {
6      let mut coeffs: Vec<u64> = polynomial.coeffs.to_vec();
7
8      let n = params.n;
9      let q = params.q as u128;
10
11     let mut t: usize = 1; //This time, t starts with the
        value of 1.
12     let mut m: usize = n; //While m starts with the value of
        n. (it's the opposite of what we do in the NTT)
13
14     loop {
15         let h = m / 2; //Number of groups during this round
16         let mut j1: usize = 0; //First bound
17
18         for i in 0..h { //For each group
19             let j2 = j1 + t - 1; //the second bound
20
21             for j in j1..j2 { //For each butterfly in the
                group
22                 let u = coeffs[j] as u128;
23                 let v = coeffs[j + t] as u128;
24
25                 let sum = u + v; //We sum then reduce (just
                    like in the NTT)
26                 let sum = if sum >= q { sum - q } else
                    { sum };
27                 coeffs[j] = sum as u64;
28
29
30                 let w = ntt_tables.twiddles_inv[h + i]; //
                    But this time, we use the inversed twiddles
31                 coeffs[j + t] = (((u + q - v) % q * w) % q)
                    as u64;
32             }
33
34             j1 += 2 * t; //We pass to the next group

```



```

35     }
36
37     t *= 2; //t is multiplied by two, while m is divided
           by two (exactly the opposite as in the NTT)
38     m /= 2;
39
40
41     if m <= 1 { break; } //When all coefficients are
           computed, we break
42 }
43
44 let n_inv = mod_pow(n as u128, q - 2, q); //Finally, we
           compute the inverse of n
45 for c in coeffs.iter_mut() {
46     *c = (*c * n_inv as u64) % q as u64; //And we scale
           each coefficient with n^(-1)
47 }
48
49 Polynomial {
50     coeffs:
           coeffs.into_iter().collect::<Vec<_>>().into_boxed_slice()
51 }
52 }

```

Explanation :

First, we know that the coeffs that are in the NTT domain (the input coefficients) are in the bit-reversed order : $\text{coeffs} = [\hat{a}_0, \hat{a}_4, \hat{a}_2, \hat{a}_6, \hat{a}_1, \hat{a}_5, \hat{a}_3, \hat{a}_7]$. We want to recover the original coefficients in standard order.

In the inverse NTT, t starts small and grows, so that :

$t=1$: $[0] \leftarrow 1 \rightarrow [1] \quad [2] \leftarrow 1 \rightarrow [3] \quad [4] \leftarrow 1 \rightarrow [5] \quad [6] \leftarrow 1 \rightarrow [7]$

$t=2$: $[0] \leftarrow 2 \rightarrow [2] \quad [1] \leftarrow 2 \rightarrow [3] \quad [4] \leftarrow 2 \rightarrow [6] \quad [5] \leftarrow 2 \rightarrow [7]$

$t=4$: $[0] \leftarrow 4 \rightarrow [4] \quad [1] \leftarrow 4 \rightarrow [5] \quad [2] \leftarrow 4 \rightarrow [6] \quad [3] \leftarrow 4 \rightarrow [7]$

It's exactly the inverse of the forward NTT.

In the INTT, m is the total covered length, while h is the number of

groupes :

$m = 8, h = 4 \Rightarrow 4$ groups of butterflies

$m = 4, h = 2 \Rightarrow 2$ groups of butterflies

$m = 2, h = 1 \Rightarrow 1$ groups of butterflies

Why $h = m/2$? Because each group contains t butterflies, so that the total number of butterflies is constant ($n/2$). So, we need :

$$h * t = n/2$$

$$h = (n/2)/t$$

$$h = m/2$$

In the INTT, j_1 and j_2 are the bounds of the group, like in the NTT, but in another way :

For a group i :

- j_1 is the sum of the length of the previous groups
- $j_2 = j_1 + t - 1$

And j is the index of u in each butterfly.

For example, with $n = 8$, at the second iteration ($t = 2, h = 2$) :

Group $i = 0$:

$$j_1 = 0$$

$$j_2 = 0 + 2 - 1 = 1$$

Butterflies on $\text{coeffs}[j] \leftrightarrow \text{coeffs}[j + t]$ and $\text{coeffs}[j] \leftrightarrow \text{coeffs}[j + t]$.

Butterflies on $\text{coeffs}[0] \leftrightarrow \text{coeffs}[2]$ and $\text{coeffs}[1] \leftrightarrow \text{coeffs}[3]$.

$$j_1 = j_1 + 2 * t = 0 + 2 * 2 = 4$$

Group $i = 1$:

$$j_2 = 4 + 2 - 1 = 5$$

Butterflies on $\text{coeffs}[j] \leftrightarrow \text{coeffs}[j + t]$ and $\text{coeffs}[j] \leftrightarrow \text{coeffs}[j + t]$.

Butterflies on $\text{coeffs}[4] \leftrightarrow \text{coeffs}[6]$ and $\text{coeffs}[5] \leftrightarrow \text{coeffs}[7]$.

...

We can visualize like this, for example :

Initial state : $\text{coeffs} = [\hat{a}_0, \hat{a}_4, \hat{a}_2, \hat{a}_6, \hat{a}_1, \hat{a}_5, \hat{a}_3, \hat{a}_7]$.

Iteration 1 : $t = 1, h = 4, m = 8$:

Group $i = 0$: $j_1 = 0, j_2 = 0 \rightarrow j = 0$: $\text{coeffs}[0] \leftrightarrow \text{coeffs}[1]$ (twiddles[4])

Group $i = 1$: $j_1 = 2, j_2 = 2 \rightarrow j = 2$: $\text{coeffs}[2] \leftrightarrow \text{coeffs}[3]$ (twiddles[5])

Group $i = 2$: $j_1 = 4, j_2 = 4 \rightarrow j = 4$: $\text{coeffs}[4] \leftrightarrow \text{coeffs}[5]$ (twiddles[6])

Group $i = 3$: $j_1 = 6, j_2 = 6 \rightarrow j = 6$: $\text{coeffs}[6] \leftrightarrow \text{coeffs}[7]$ (twiddles[7])

4 groups of 1 butterfly, distance of 1

Iteration 2 : $t = 2, m = 2$:

Group $i = 0$: $j_1 = 0, j_2 = 1$:

$j=0$: $\text{coeffs}[0] \leftrightarrow \text{coeffs}[2]$ (twiddle = twiddles[2])

$j=1$: $\text{coeffs}[1] \leftrightarrow \text{coeffs}[3]$ (twiddle = twiddles[2])

Group $i = 1$: $j_1 = 4, j_2 = 5$

$j=4$: $\text{coeffs}[4] \leftrightarrow \text{coeffs}[6]$ (twiddle = twiddles[3])

$j=5$: $\text{coeffs}[5] \leftrightarrow \text{coeffs}[7]$ (twiddle = twiddles[3])

2 groups of 2 butterflies, distance of 2

Iteration 3 : $t = 4, h = 1$

Group $i = 0 : j_1 = 0, j_2 = 3$

j=0 : coeffs[0] $\leftrightarrow$ coeffs[4] (twiddle = twiddles[1])

j=1 : coeffs[1] $\leftrightarrow$ coeffs[5] (twiddle = twiddles[1])

j=2 : coeffs[2] $\leftrightarrow$ coeffs[6] (twiddle = twiddles[1])

j=3 : coeffs[3] $\leftrightarrow$ coeffs[7] (twiddle = twiddles[1])

4 groups of 1 butterfly, distance of 1

Result : coeffs = $[a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7]$ (normal order)

Why the twiddles_inv[h + i] ? Because the access to the twiddles follows the structure of the groups (see above).

Finally, you can see that the inverse NTT is the exact implementation of the mathematical formula ($\omega^{-i \cdot j}$ in the butterflies and n^{-1} at the end), and the exact opposite of the computation of butterflies (opposite to the computation in the forward NTT).

Inverse NTT in multi-thread

We can now also implement a parallelized version of the inverse NTT algorithm :

```
1  #[cfg(feature = "parallel")] Rust
2  fn inverse_ntt_multi(
3      params: &RingParams,
4      polynomial: &Polynomial,
5      ntt_tables: &NTTprecalculated
6  ) -> Polynomial {
7      let mut coeffs: Vec<u128> = polynomial.coeffs
8          .iter()
9          .map(|&c| c as u128)
10         .collect();
11
12     let n = params.n;
13     let q = params.q as u128;
```

```

14
15     let mut t: usize = 1; //We use the same parameters, the
    same base.
16     let mut m: usize = n;
17
18     loop {
19         let h = m / 2;
20
21         coeffs
22         .par_chunks_mut(2 * t) //We "cut" this time also by
    chunks of 2t
23         .enumerate()
24         .for_each(|(i, chunk)| {
25             let w = ntt_tables.twiddles_inv[h + i]; //Access to
    the inversed twiddles
26
27             for j in 0..t { //Exactly the same
28                 let u = chunk[j];
29                 let v = chunk[j + t];
30
31                 let sum = u + v;
32                 chunk[j] = if sum >= q { sum - q } else { sum };
33
34                 chunk[j + t] = ((u + q - v) % q * w) % q;
35             }
36         });
37
38         t = t * 2;
39         m = m / 2;
40
41
42         if m <= 1 { break; }
43     }
44
45     let n_inv = mod_pow(n as u128, q - 2, q); //Final
    scaling
46     for c in coeffs.iter_mut() {
47         *c = (*c * n_inv) % q;

```

```

48     }
49
50     Polynomial {
51         coeffs: coeffs.into_iter().map(|c| c as
52         u64).collect::<Vec<_>>().into_boxed_slice(),
53     }
54 }

```

Polynomial multiplication with NTT

We can finally implement what we need : the polynomial multiplication using the NTT, which is much more efficient than the naive multiplication. This function is really simple, since we already have done the hard job by implementing the NTT :

```

1  #[inline] 🔒 Rust
   pub fn mul_ntt(&self, params: &RingParams, ntt_tables:
2  &NTTprecaculated, polynomial: &Polynomial) -> Polynomial
   { //Function that computes the product of two polynomials by
     passing them in NTT domain.
3
     let a = forward_ntt(params, self, ntt_tables); //First,
     we pass the polynomials into the NTT domain
4
     let b = forward_ntt(params, polynomial, ntt_tables);
5
6
     let mut c_ntt = Polynomial::zeros(params.n);
7
8
     for i in 0..params.n {
9
         c_ntt.coeffs[i] = ((a.coeffs[i] as u128 *
         b.coeffs[i] as u128) % params.q as u128) as u64; //
         Then, we just do a basic pointwise multiplication
10    }
11
12    inverse_ntt(params, &c_ntt, ntt_tables) //And we finally
     get back into the "normal" domain
13
14 }

```

Errors

In `simple-ring`, I didn't implement Errors in a `errors.rs` file, with an enum (`RingError` for example), and I didn't make all my functions return a `Result<Polynomial, RingError>`. Why ? Because it would have added more complexity. This implementation has to remain simple to use, as :

```
1 let params = RingParams::new(16, 17, find_valid_omega(16, 17));  
2 let ntt_tables = &precalculate(&params);  
3 let poly = Polynomial::new(vec![2u64; params.n]);  
4 let poly2 = Polynomial::new(vec![4u64; params.n]);  
5 let product = poly.mul_ntt(&params, ntt_tables, &poly2);
```

And not :

```
1 let params = RingParams::new(16, 17, find_valid_omega(16, 17));  
2 let ntt_tables = &precalculate(&params);  
3 let poly = Polynomial::new(vec![2u64; params.n]);  
4 let poly2 = Polynomial::new(vec![4u64; params.n]);  
5 let product = poly.mul_ntt(&params, ntt_tables, &poly2).unwrap();
```

Nor

```
1 fn main() -> Result<(), RingError> {  
2 let params = RingParams::new(16, 17, find_valid_omega(16, 17));  
3 let ntt_tables = &precalculate(&params);  
4 let poly = Polynomial::new(vec![2u64; params.n]);  
5 let poly2 = Polynomial::new(vec![4u64; params.n]);  
6 let product = poly.mul_ntt(&params, ntt_tables, &poly2)?;  
7 }
```

So, I decided to add `assert_eq!()` in the code, to prevent silent errors. In the case where something is wrong, the code should panic, and explain in details what is the error, without needing a `?` nor an `unwrap()`.

Glossary

Term	Definition
R_q	Polynomial ring $\mathbb{Z}_q[X]/(X^n + 1)$
NTT	Number Theoretic Transform: FFT over finite fields
CBD	Centered Binomial Distribution: discrete approximation of Gaussian
Twiddle	Precomputed power of omega: $t_k = \omega^k \bmod q$

References

- [1] : Özcan, Ali & Ayduman, Can & Türkoğlu, Enes & Savaş, Erkey. (2023). Homomorphic Encryption on GPU. IEEE Access. PP. 1-1. 10.1109/ACCESS.2023.3265583. (<https://eprint.iacr.org/2022/1222.pdf>)
- [2] : Sengupta, B., Gupta, P., & Sengupta, S. (2025). Introduction to Number Theoretic Transform. ArXiv, abs/2509.05884. (<https://arxiv.org/pdf/2509.05884>)
- [3] : Damien Stehlé. Euclidean lattices: algorithms and cryptography. Cryptography and Security [cs.CR]. Ecole normale supérieure de lyon - ENS LYON, 2011. <tel-00645387>. (<https://theses.hal.science/tel-00645387/>)
- [4] : Fermat's Little theorem, on Wikipedia (https://en.wikipedia.org/wiki/Fermat%27s_little_theorem)
- [5] : Ring, on Wikipedia ([https://en.wikipedia.org/wiki/Ring_\(mathematics\)](https://en.wikipedia.org/wiki/Ring_(mathematics)))
- [6] : Avanzi, Roberto. CRYSTALS-Kyber Algorithm Specifications And Supporting Documentation. 2017. (<https://pq-crystals.org/kyber/data/kyber-specification-round3-20210131.pdf>)
- [7] : S. Lee, J. Woo, J. Kim and J. H. Park, "Generalized Centered Binomial Distribution for Bimodal Lattice Signatures," in IEEE Access, vol. 13, pp. 2203-2214, 2025, doi: 10.1109/ACCESS.2024.3523521. keywords: {Gaussian distribution;Standards;Polynomials;Digital signatures;Lattices;Timing;Random variables;Newton method;Hypercubes;Cryptography;Post-quantum cryptography;lattice-based signatures;bimodal;binomial distribution}, (<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10817596>)